

Documentation technique

Data Lake financier

Groupe : Artemiy Smogunov, Nathan Smadja-Tubiana, Patrice Ignongui

Projet EFREI 2025-2026

Financial Data Lake (Projet EFREI 2025-2026)

Projet final du cours *Data Lakes & Data Integration*.

Groupe

- Artemiy Smogunov
- Nathan Smadja-Tubiana
- Patrice Ignongui

Lien du sujet

- Sujet : Data_Lakes_Projet_Final_EFREI_2025-2026.pdf
- Récapitulatif des règles : resume_regles_devoir.txt

Ce que couvre ce dépôt

Ce projet met en place un mini data lake financier avec deux sources de données, trois zones (Raw / Staging / Curated), une API d'exposition, et une orchestration planifiée.

- Sources utilisées :
- data/finance_dataset.csv (fichier local)
- API Yahoo Finance via yfinance
- Zone Raw : MinIO (S3 compatible) et Elasticsearch
- Zones Staging et Curated : PostgreSQL
- Orchestration : Apache Airflow
- API : FastAPI

Architecture (vue courte)

```
Sources (CSV local, yfinance)
|-- ingestion_file.py --|
`-- ingestion_api.py ---+--> Raw : MinIO (objets CSV/JSON) + Elasticsearch
                        |--> Airflow DAG
                        |
                        v
                    Staging : PostgreSQL (staging_ohlcv)
                        |
                        v
                    Curated : PostgreSQL (curated_analysis)
                        |
                        v
                    FastAPI : /raw, /staging, /curated, /health, /stats
                        |
                        |--> endpoints avancés /ingest, /ingest_fast
```

Correspondance avec le devoir

| Exigence | Mise en œuvre | Preuve principale |

|---|---|---|

| Conception d'un data lake complet | ingestion/, transformation/, api/, airflow/ | README + code + scripts SQL |

| Structure Raw / Staging / Curated | ingestion/, transformation/staging/, transformation/curated/ | scripts/init_db.sql, requêtes de api/routers |

| Stockage raw compatible S3 + Elasticsearch | ingestion/ingest_file.py, ingestion/ingest_api.py, docker-compose.yml, api/dependencies.py | index/métadonnées dans ES, objets MinIO |

| Ingestion depuis 2 sources | ingestion/ingest_file.py, ingestion/ingest_api.py, config/settings.py | ALL_TICKERS + pipeline d'ingestion |

| Pipeline d'intégration | airflow/dags/financial_pipeline_dag.py | tâches parallèle file/API puis staging puis curated |

| API Gateway obligatoire | api/main.py, api/routers/* | endpoints /raw, /staging, /curated, /health, /stats |

| Niveau avancé /ingest + /ingest_fast + benchmark | api/routers/ingest.py, api/routers/ingest_fast.py, scripts/benchmark_endpoints.py | script de benchmark reproductible |

Stack et ports

| Élément | Rôle | Port | Lieu |

|---|---|---|

| PostgreSQL | Staging, Curated, logs d'ingestion | 5432 | docker-compose.yml |

| MinIO API | Stockage objets Raw | 9000 | docker-compose.yml |

| MinIO Console | UI de buckets | 9001 | docker-compose.yml |

| Elasticsearch | Index brut | 9200 | docker-compose.yml |

| Airflow UI | Orchestration DAG | 8080 | airflow services |

| FastAPI | API Gateway | 8000 | api/ service |

Prérequis

- Docker + Docker Compose
- Git (pour la reproduction du dépôt)

Installation et lancement

```
git clone https://github.com/Nathan2412/Projet-DataLake-Final.git
cd Projet-DataLake-Final
```

```
docker compose up -d
```

Attendre le démarrage puis vérifier l'état :

```
docker compose ps
docker compose logs api
```

Services attendus : PostgreSQL, MinIO, Elasticsearch, Airflow Webserver, Airflow Scheduler, API.

Vérification rapide après démarrage

```
curl http://localhost:8000/health
curl http://localhost:8000/docs
curl http://localhost:8080
curl http://localhost:9001
```

Endpoints attendus

- GET /health : vérifie Postgres, MinIO et Elasticsearch.
- GET /stats : métriques par zone.
- GET /raw : documents bruts depuis Elasticsearch avec filtres (ticker, source, dates, limit).
- GET /raw/objects : objets présents dans MinIO.
- GET /staging : OHLCV nettoyé et indicateurs.
- GET /staging/tickers : liste des tickers disponibles.
- GET /curated : anomalies et signaux de trading.
- GET /curated/anomalies/summary : résumé des anomalies par ticker/type.
- GET /curated/signals : tickers avec signal actif buy ou sell.
- POST /ingest : pipeline standard séquentiel.
- POST /ingest_fast : pipeline parallèle.

Appels d'ingestion utiles

Ingestion standard

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "data": {
    "tickers": ["AAPL", "MSFT", "^GSPC"],
    "period": "1mo",
    "run_staging": true,
    "run_curated": true
  }
}'
```

Ingestion parallèle

```
curl -X POST http://localhost:8000/ingest_fast \
-H "Content-Type: application/json" \
-d '{
  "data": {
    "tickers": ["AAPL", "MSFT", "^GSPC"],
    "period": "1mo",
    "run_staging": true,
  }
}'
```

```
        "run_curated": true
    }
}'
```

Les deux endpoints retournent une structure avec status, pipeline_steps, performance, errors, timestamp.

Comment vérifier les données manuellement

```
curl "http://localhost:8000/staging?ticker=AAPL&limit=20"
curl "http://localhost:8000/curated?ticker=AAPL&anomalies_only=true&limit=20"
curl "http://localhost:8000/stats"
```

Tests exécutables localement (sans stack externe)

```
python -m pytest tests/test_financial_logic.py tests/test_benchmark.py
```

Résultat observé sur cette branche : 12 tests passants.

Benchmark reproductible

Le script enregistre systématiquement un rapport JSON, pour batch 1 et 100.

```
python scripts/benchmark_endpoints.py \
  --base-url http://localhost:8000 \
  --period 5d \
  --repeats 3 \
  --timeout 1800 \
  --output livrables/benchmark_ingest_vs_ingest_fast.json
```

Résultats du test du 10 juillet 2026, avec trois répétitions par lot :

- 1 ticker : /ingest 1 073,29 ms, /ingest_fast 989,86 ms, gain de 7,77 %.
- 100 tickers : /ingest 104 825,65 ms, /ingest_fast 12 230,36 ms, gain de 88,33 %.

La cible de 30 % est atteinte sur le lot de 100 tickers. Les douze appels ont réussi sans erreur. Le fichier JSON contient les échantillons, l'ordre des appels et le commit testé 85aa820.

Limites connues

- Les temps d'ingestion varient selon la disponibilité réseau Yahoo Finance.
- Le benchmark doit être relancé après une modification des pipelines ; le résultat commité décrit uniquement le commit indiqué dans le JSON.

Régénérer les PDF

```
.venv/bin/pip install -r scripts/requirements-docs.txt
.venv/bin/python scripts/generate_pdf_deliverables.py
```

Livrables attendus

- livrables/Rapport_DataLake_Finance_Artemiy_Smogunov_Nathan_Smadja-Tubiana_Patrice_Ignongui.pdf
- livrables/Documentation_Technique_DataLake_Finance_Artemiy_Smogunov_Nathan_Smadja-Tubiana_Patrice_Ignongui.pdf
- livrables/benchmark_ingest_vs_ingest_fast.json (mesure finale reproductible)

Conformité du devoir

Sujet : EFREI Data Lakes & Data Integration 2025-2026

Vérifications réalisées le 10 juillet 2026 sur la stack Docker.

Exigences obligatoires

1. Data Lake de bout en bout

Le projet couvre l'ingestion, le stockage, les transformations et l'exposition par API. Les tests passent et les endpoints répondent.

2. Zones Raw, Staging et Curated

Raw utilise MinIO et Elasticsearch. Staging et Curated utilisent PostgreSQL.

3. Deux sources de données

Le pipeline utilise `data/finance_dataset.csv` et Yahoo Finance via `yfinance`. Les deux sources sont visibles dans la zone Raw.

4. Pipeline Airflow

Le DAG `financial_data_lake_pipeline` a été exécuté avec l'identifiant `manual_final_20260710T132822Z`. Les sept tâches se sont terminées avec le statut `success`.

5. API Gateway

Les endpoints `/raw`, `/staging`, `/curated`, `/health` et `/stats` ont été testés sur la stack Docker.

6. Livrables

Le dépôt contient le README, le rapport technique, la documentation technique, les deux PDF et le résultat du benchmark.

Niveau avancé

Les endpoints `/ingest` et `/ingest_fast` ont été testés.

Le benchmark compare des lots de 1 et 100 tickers avec trois répétitions et un ordre alterné. Les douze appels ont réussi.

Pour 100 tickers, le temps médian passe de 104 825,65 ms à 12 230,36 ms. Le gain mesuré est de 88,33 %, supérieur à la cible de 30 %.

Les mesures sont enregistrées dans `livrables/benchmark_ingest_vs_ingest_fast.json`. Le fichier indique le commit testé `85aa820` et contient chaque échantillon.

Commandes de vérification

```
python -m pytest tests/test_financial_logic.py tests/test_benchmark.py
docker compose up -d
curl http://localhost:8000/health
curl http://localhost:8000/stats
curl "http://localhost:8000/raw?limit=5"
curl "http://localhost:8000/staging?limit=5"
curl "http://localhost:8000/curated?limit=5"
python scripts/benchmark_endpoints.py --base-url http://localhost:8000 --period 5d --repeats 3
```